ATLAS-ASR-001

Actual System Review — Version 2

Project	ATLAS Master Flightline Clock
Document ID	ATLAS-ASR-001
Version	2.0
System Version	v1.6.0
Date	July 2026
Status	CURRENT AUTHORITATIVE — supersedes v1
Review Basis	Direct inspection of uploaded source files and July 2026 handover dossier

1. Purpose

This document records the actual implemented state of the ATLAS system based on direct inspection of the live codebase and associated project materials as of July 2026. It supersedes the previous actual system review, which reflected an earlier and now outdated system state.

2. Executive Summary

ATLAS is no longer a conceptual or early-foundation project. It is a functioning desktop application with operational scheduling, timing, persistence, and display capabilities. The implementation is sufficiently mature to support controlled deployment preparation work.

The system is best described as a **working, dual-backend race timing appliance** with an operator console, SQLite-backed persistence, and emerging appliance-style startup behaviour.

> The system runs successfully on the developer's Mac (Python 3.12, PySide6 6.11.1) and has been verified operational.

3. What Is Working

Feature	Status	Notes
Real-time local clock display	**WORKING**	1Hz sync to system clock

Heat scheduling and persistence	**WORKING**	SQLite-backed
Operator heat add / edit / delete	**WORKING**	Full CRUD via Schedule tab
Automatic heat activation	**WORKING**	Activates within 15-minute threshold
Phase-aware countdown display	**WORKING**	Yellow / Green / Red phase logic
Operator heat cancellation	**WORKING**	ABORT button on LIVE tab
Settings persistence	**WORKING**	All settings in SQLite
Brightness control	**WORKING**	Live signal update, no restart needed
LED simulator display backend	**WORKING**	128x64 and 64x64 layouts
LCD monitor display backend	**WORKING**	Full-resolution, anti-aliased
Display backend selection	**WORKING**	Settings tab dropdown, requires restart
Auto-boot display-only mode	**WORKING**	Checkbox in Settings, requires restart
Auto-boot checkbox styling	**WORKING**	Golden border, prominent styling confirmed
Restart ATLAS function	**WORKING**	os.execv process replacement
Exit application function	**WORKING**	QApplication.quit() — all windows

4. Actual Implemented Architecture

4.1 main.py

The current `main.py` performs:

- Application bootstrap with Fusion style for cross-platform consistency
- Database initialisation
- Engine construction
- Dynamic display configuration loading from database
- Backend selection via factory function
- Auto-boot startup branching (display-only vs full console)

4.2 database.py

The current `database.py` provides:

- Schema bootstrap with `CREATE TABLE IF NOT EXISTS`
- Backward-compatible seeding of newer settings via `INSERT OR IGNORE`

- Full heat CRUD operations
- Settings get/set operations
- Boolean setting coercion via `get_bool_setting()`

4.3 windows.py

The current `windows.py` includes:

- Three-tab operator console: LIVE, SCHEDULE, SETTINGS
- Display backend dropdown (LED Simulator / LCD Monitor)
- Display size dropdown (128x64 / 64x64)
- Brightness slider with live preview and named labels
- Auto-boot checkbox with prominent golden styling
- SAVE & APPLY, RESTART ATLAS, EXIT TO MACOS buttons

4.4 Display Backends

`led_simulator.py` and `lcd_display.py` form a dual-backend rendering model. Both implement identical public interfaces and are interchangeable via the factory in `main.py`.

5. What Is Outdated in Earlier Reviews

> The following findings from earlier system reviews are now superseded and should not be treated as current:

Earlier Finding	Current Reality
ATLAS is simulator-only	LCD monitor backend now exists and is selectable
No backend selection mechanism	Backend selection is fully implemented in Settings tab
No auto-boot support	Auto-boot setting and startup branching are implemented
Checkbox not visible / poorly styled	Prominent golden styling confirmed working
fonts.py missing letter 'X'	Incorrect — user's actual fonts.py is complete and correct
JSON config is primary settings source	SQLite is the authoritative runtime settings source

6. Confirmed Legacy / Vestigial Items

Item	Status	Notes
`src/core/config.py`	**LEGACY**	Not imported by active code. JSON-based config from earlier architecture.
`src/core/timers.py`	**LEGACY**	Not imported by active code. Standalone timer class from pre-v0.8.0 architecture.
`atlas_config.json`	**LEGACY**	Not read by current main.py. Retained for historical continuity.

7. Remaining Gaps and Technical Debt

Item	Risk Level	Recommended Action
Database path uses relative `data/atlas.db`	**MEDIUM**	Harden to project-relative absolute path before deployment
Startup scripts are macOS-specific	**MEDIUM**	Create Windows equivalents
UI wording "EXIT TO MACOS"	**LOW**	Generalise for cross-platform deployment
Legacy modules in-tree	**LOW**	Document as legacy or formally retire
closeEvent only prints to console	**LOW**	Placeholder for future hardware clear-screen command
Windows deployment not yet validated	**MEDIUM**	Deployment process to be defined and tested

8. Conclusion

ATLAS is in a practical **"working system under refinement"** state rather than a conceptual or prototype-only state. The codebase is clean, the architecture is coherent, and the system runs reliably on the development machine.

The next sensible engineering steps are deployment hardening and platform packaging — not architectural reinvention.

> No significant architectural changes are recommended at this time. The system is ready for deployment preparation work.